

# Coverage at a Finite Budget: Filling the Reachable Behavior Space with $n = 10,000$ Items

## 1 Abstract

Part I asks how to generate an unbounded stream of items that never collapses onto its own past, under a packing objective: keep every new item far from everything already made. This part studies the covering objective. A fixed budget of  $n = 10,000$  items must be spent so that the resulting set *covers* as much of the generator’s reachable behavior space as possible: we maximize the measure of the union of small  $\epsilon$ -balls centered at the chosen embeddings, estimated by Monte Carlo against a reference pool of cheap draws from the generator itself. The coverage functional is monotone submodular, so greedy marginal-gain selection over a candidate stream inherits the classical  $(1 - 1/e)$  guarantee, and its Monte-Carlo estimate concentrates at the Hoeffding rate; we verify all three facts numerically rather than taking them on faith (0 violations of diminishing returns in 5,000 randomized chain trials; greedy attains the exact optimum on all 10 enumerable instances; the empirical 95% error of the pool estimate sits inside the Hoeffding band at every pool size tested). We also measure a *reachability gap* — the 17.4 points of coverage that no selection rule can recover because the proposal distribution cannot reach them, 92.5% of which recursive refinement does recover, which follows from having an embedding oracle but no inverse. The estimator is validated against analytically known answers, and evaluation is budget-matched on real gpt-5.6-luna corpora with a stratum-balanced reference and a scoring half held out from the selector. Coverage-greedy selection beats all five literature baselines — SemDeDup, farthest-point/k-center, Self-Instruct’s ROUGE-L filter, greedy MAP-DPP, and uniform random — by 1.043x on DALL-E instructions and 1.091x on psychometric items. Against the released corpora (Alpaca, PersonaHub, WizardLM), covered fraction at a fixed radius is not comparable across corpora with different intrinsic scales — it correlates -0.991 with within-corpus spacing and -0.986 with Vendi — so we use a scale-free estimator with reference-side k-NN radii. On it, coverage-retrieval conditioning places first of twelve corpora at 0.4441 against Alpaca’s 0.3722, at one-twentieth of Alpaca’s generation budget and with the highest precision in the field (0.973). The same data shows the covering/packing dissociation at full strength: k-center maximizes min-gap (0.696), and finishes last on coverage (0.096, a fifth of random), and the two highest-Vendi selectors are the two worst covering ones. Naive psychometric prompting is 73.6% exact duplicates (one item repeated 2,726 times in 10,000 generations), which high temperature barely dents (71.0%) and conditioning nearly eliminates (0.0-4.7%) — direct evidence that conditioning, not temperature, buys coverage, and a demonstration that nearest-neighbor epsilon-calibration is undefined on such a corpus (median self-distance exactly 0). Finally, literal and latent diversity move in opposite directions as the corpus grows (distinct-2 falls  $0.944 \rightarrow 0.770$  while mean-centered Vendi rises  $3.98 \rightarrow 37.06$ ), and the uncentered Vendi is compressed  $2.7\times$  by the embeddings’ shared mean direction — so both levels, and both centerings, need reporting.

## 2 Introduction

A team building an evaluation suite for a language model does not get an infinite horizon. They get a budget (ten thousand prompts, say), and a product requirement that is naturally a *coverage* requirement: the suite should contain an example near every qualitatively distinct behavior the system under test might encounter, because a suite that clusters into a few families gives false assurance about everything outside them. The same shape of problem appears when generating synthetic user personas to exercise a dialogue system: near-duplicate personas waste budget, and bugs live in the parts of user-behavior space the corpus never visits.

The parent paper in `research/infinite_horizon_diversity/` (this work lives in its `coverage/` subdirectory and imports its modules directly) studies the open-ended version of this problem: an unbounded stream, an objective that combines typicality (mean distance to prior embeddings) with max-min novelty, and a method stack — generator-elicited latent axes, spectral orthogonalization against the corpus second-moment matrix, recursive refinement of saturated cells, and an append-only attractor ledger — that keeps the stream from collapsing at any horizon. This paper keeps the method stack and swaps the objective, because the budgeted problem is *not* the truncation of the infinite one:

1. **Max-min packing is the wrong objective at a finite budget.** Packing pushes points apart; its optima sit on the boundary of the reachable region and over-invest in extremes and outliers, because an outlier is by construction far from everything. Coverage weights regions by how much probability mass they hold and fills the bulk first. The two objectives correspond to two different classical problems — k-center versus maximum coverage / facility location, and their optima genuinely differ (§3.4, §7.2).
2. **A known  $n$  permits planning.** With an unbounded horizon there is nothing to allocate; with  $n = 10,000$  known in advance one can ask how the budget should be split across cells of the behavior space — equally, proportionally to measure, or adaptively, and the answer depends on the radius  $\varepsilon$  at which coverage is demanded (§3.5, §7).
3. **Coverage needs a denominator.** “We covered 62% ” is meaningless without saying 62% *of what*. In the pipeline the denominator is the generator’s own reachable distribution, approximated by a pool of cheap draws. For comparing corpora built by different methods it is a held-out human-written reference under a scale-free estimator whose radii are a property of the reference alone — fixed, identical for every corpus scored, and impossible for the method being graded to grow (§5).

Everything is built to the same scaling discipline as Part I: per-item cost is  $O(1)$  in  $n$  (fixed-size reference pool, incremental covered masks, bounded history subsamples), so nothing in the pipeline gets slower at item 10,000 than at item 100.

We make four contributions. First, a head-to-head against released instruction corpora — Alpaca, WizardLM Evol-Instruct, Persona-Hub — scored by a scale-free coverage estimator against a held-out human-written reference no corpus was aimed at, which our method wins at matched evaluated  $n$  on a twentieth of the generation budget (§5). Second, a coverage adaptation of Part I’s axis-scoring rule in which the four-factor product collapses to a single estimable scalar (the expected marginal  $\varepsilon$ -ball gain of conditioning on the axis) with an ablation isolating what score-guided conditioning buys (§4, §5). Third, a live pilot of the full method (elicited axes, score-guided focus, judge gate, ledger, refinement, coverage-greedy selection) on gpt-5.6-luna with local embeddings (§6). Fourth,

a formalization of budgeted corpus construction as Monte-Carlo maximization of a union-of-balls coverage functional, with the relevant guarantees stated precisely and *checked numerically* (§3, §7), including the no-inverse-oracle propositions and a direct measurement of the reachability gap they imply (§2, §7.6).

### 3 Problem statement

Fix an embedding map  $\varphi$  into  $R^D$  (we use unit-normalized 768-d embeddings), and a radius  $\varepsilon > 0$ . The generator, prompted in whatever ways the pipeline can reach, induces a *reachable distribution*  $\mu$  over  $R^D$ : the pushforward of everything the generator can actually be made to produce. Given a budget  $n$ , choose a set  $S = \{x_1, \dots, x_n\}$  of generated items to maximize the covered measure

$$F_{\mu, \varepsilon}(S) = \mu\left(\bigcup_{x \in S} B(x, \varepsilon)\right),$$

the probability that a fresh draw from the generator lands within  $\varepsilon$  of some chosen item. Equivalently,  $S$  should be an  $\varepsilon$ -net for as much of  $\mu$  as the budget allows. Three deliberate choices:

- **Coverage of the reachable space, not of  $R^D$ .** Balls in empty space cover Lebesgue volume but no behavior. Measuring against  $\mu$  makes the metric mean “fraction of what the generator can do that the corpus has an exemplar of”, which is the product requirement for an eval suite. The denominator is therefore stated next to every coverage number; where corpora built by different methods are compared, it is a held-out human-written reference identical for all of them (§5).
- **$\varepsilon$  is a resolution parameter, not a nuisance.** Small  $\varepsilon$  asks for exemplars of fine behavioral distinctions; large  $\varepsilon$  only for one exemplar per coarse region. Every result below names the radius, or the range of radii, it is reported at, because the ranking of methods changes with  $\varepsilon$ .
- **Quality is a constraint, not part of the objective.** An item that covers new territory but fails a quality/typicality bar is not an asset, so the judge gate rejects it regardless of the gain it would have scored (§4).

**The oracle asymmetry.** One assumption structures everything downstream: we have an embedding oracle  $E$ :  $\text{text} \rightarrow R^D$ , and *no inverse*. There is no  $E^{-1}$  that decodes a target point back into text. We can compute exactly where in embedding space the next item should land — the largest uncovered region, the trailing eigenspace, but we cannot manufacture an item that lands there. The only handles a pipeline has are (a) language-valued latent conditioning that *steers* the generator’s proposal distribution, and (b) selection among sampled candidates. This is why the architecture is propose–measure–select rather than solve, and it has two consequences we state as propositions:

**Proposition 1** (no direct construction). *Coverage-optimal point sets are not directly constructible. The greedy  $(1 - 1/e)$  guarantee (§3.2) holds relative to the best subset of the proposal distribution’s support, not of  $R^D$ ; the difference between what an unconstrained selector could cover and what a proposal-limited selector can is a reachability gap, which is a property of the generator-plus-conditioning stack, not of the selection algorithm. We measure it directly in §7.6.*

**Proposition 2** (conditioning as surrogate inverse). *Language-valued conditioning is the only available surrogate for  $E^{-1}$ : choosing which axis to condition on next is choosing which slice of the reachable manifold the next candidates will be sampled from, so a axis scoring that ranks axes by where their slices land (§4) is precisely an approximate inverse — it maps “where we want mass” to “which words to condition on”.*

## 4 Theory

### 4.1 The coverage functional is monotone submodular

For any measure  $\mu$  and radius  $\varepsilon$ , define  $F(S) = \mu(\bigcup_{x \in S} B(x, \varepsilon))$  on finite  $S \subset R^D$ .  $F$  is monotone (adding a ball cannot uncover anything) and submodular: for  $S \subseteq T$  and any  $x$ ,

$$F(S \cup x) - F(S) \geq F(T \cup x) - F(T)$$

*Proof sketch.* The marginal gain of  $x$  given  $S$  is  $\mu(B(x, \varepsilon) \setminus \bigcup_{y \in S} B(y, \varepsilon))$ . Since  $S \subseteq T$  implies  $\bigcup_{y \in S} B(y, \varepsilon) \subseteq \bigcup_{y \in T} B(y, \varepsilon)$ , the set being subtracted only grows, so the gain only shrinks. This is the standard coverage-function argument; nothing about balls is essential — any map from items to measurable “footprints” yields a submodular  $F$ .  $\square$

The same argument applies verbatim to the Monte-Carlo estimator we actually optimize. Given a reference pool  $P = \{p_1, \dots, p_P\}$  of iid draws from  $\mu$ ,

$$\hat{F}(S) = (1/P) \cdot \#p \in P : \text{dist}(p, S) \leq \varepsilon$$

is itself a finite coverage function (item  $x$  covers the fixed subset  $N_\varepsilon(x) = \{p : \|p - x\| \leq \varepsilon\}$  of the pool, and  $\hat{F}(S) = |\bigcup_{x \in S} N_\varepsilon(x)|/P$ ) so  $\hat{F}$  is monotone submodular exactly, at every sample of the pool. Our implementation check (§6.1) found 0 violations of either property in 5,000 randomized nested-chain trials, as it must if the code is right.

### 4.2 Greedy and its guarantee

Because  $\hat{F}$  is monotone submodular with  $\hat{F}(\emptyset) = 0$ , the greedy algorithm that repeatedly adds the item of maximum marginal gain satisfies the Nemhauser–Wolsey–Fisher bound

$$\hat{F}(S_{\text{greedy}}) \geq (1 - 1/e) \cdot \max_{|S| \leq n} \hat{F}(S) \approx 0.632 \cdot \text{OPT},$$

and this is tight for the class (maximizing coverage is NP-hard, and beating  $1 - 1/e$  is hard under standard assumptions, Feige 1998). §7.3 checks the bound against *exact* optima on instances small enough to enumerate; greedy attained the optimum itself on all ten instances, comfortably above the bound — typical behavior, since the  $1 - 1/e$  worst case requires adversarial structure.

### 4.3 Sequential generation is streaming greedy

A generation pipeline cannot pick from a precomputed ground set: candidates arrive a few at a time ( $K$  per step), and must be accepted or discarded immediately. This is the streaming submodular maximization regime. Sieve-streaming (Badanidiyuru, Mirzasoleiman, Karbasi, Krause 2014)

maintains geometric thresholds and accepts any arrival whose marginal gain clears one, achieving a  $(1/2 - \delta)$  guarantee in one pass with  $O((k \log k)/\delta)$  memory, independent of stream length. Our per-step rule (take the best of  $K$  fresh candidates by marginal gain) is the natural best-of- $K$  relaxation: weaker in the worst case (an adversarial stream can starve it) but stronger on exchangeable streams like ours, where each batch is iid from the current proposal distribution and thresholding would only discard budget. Empirically the  $K = 4$  stream rule retains most of full greedy’s value over an identical ground set (0.484 vs 0.528 covered fraction; §6.2) at a per-item cost that never touches the whole ground set. What matters for scaling is that the marginal-gain oracle is  $O(K \cdot P)$  per step against a fixed-size pool with an incrementally maintained covered mask —  $O(1)$  in  $n$ .

#### 4.4 Coverage is not packing

Max-min selection (choose the candidate farthest from the accepted set) is the greedy algorithm for the  $k$ -center / packing objective, and Part I paper shows it is the right *shape* of objective for an unbounded stream once a typicality anchor keeps it on-manifold. At a finite budget against a fixed  $\mu$  the two objectives pull apart:

- $k$ -center cares about the worst-covered *point of  $S$  itself*; its optima push to the boundary of the support and place points in regions of vanishing measure (an outlier is far from everything, so packing *rewards* it), and under a fixed budget every such item is a ball spent covering  $\approx 0$  measure.
- Maximum coverage / facility-location-type objectives weight territory by measure: a second exemplar in a heavy mode can be worth more than a first exemplar of a negligible one, and junk is worth nothing because  $\mu$  puts almost nothing near it.

§6.2 isolates the contrast with no generator in the loop: on one shared candidate set, greedy-coverage covered 0.528 of a held-out pool where farthest-point packing covered 0.143, while packing’s min-gap (1.44) beat greedy’s (0.577) by  $2.5\times$ . Neither is “better”; they optimize different functionals, and §7.2 shows the same double dissociation with the full generation loop in place.

#### 4.5 Estimation error of the Monte-Carlo coverage

For fixed  $S$ , each pool point contributes an iid Bernoulli( $F(S)$ ) indicator, so Hoeffding/Chernoff gives

$$P(|\hat{F}(S) - F(S)| > t) \leq 2\exp(-2Pt^2),$$

i.e. a 95% band of  $t_{95} = \sqrt{\ln(2/0.05)/(2P)}$  — about  $\pm 0.030$  at  $P = 2,000$  and  $\pm 0.015$  at  $P = 8,000$ . Two caveats we take seriously. First, the bound is for fixed  $S$ ; a set *selected* by optimizing  $\hat{F}$  on the same pool is biased upward on that pool (the winner’s curse), which is why every coverage number reported here is computed on held-out material the selector never saw, and the live pilot reports selection-pool coverage explicitly labeled as such. Second, uniform-over- $S$  guarantees would need a union bound over the (exponentially many) candidate sets; we do not rely on one — held-out evaluation makes the fixed- $S$  bound the relevant one. §7.4 confirms the band empirically: at every pool size tested,  $\geq 99\%$  of 200 independent pool estimates fell within  $t_{95}$  of a 200k-pool ground truth.

## 4.6 Allocation under a known budget

Suppose the space decomposes into cells (modes) with measures  $w_m$ , and within-cell placement is near-saturating: with  $c_m$  items in cell  $m$ , the covered measure of that cell is approximately  $w_m \cdot g_m(c_m)$  with  $g_m$  increasing and concave (each additional exemplar covers a shrinking uncovered remainder — submodularity again, now per cell). Maximizing  $\sum w_m g_m(c_m)$  subject to  $\sum c_m = n$  is a concave allocation problem whose optimum is the water-filling rule: equalize marginal gains  $w_m g'_m(c_m)$  across cells. Three closed-form rules bracket the regimes:

- **$\varepsilon$  large relative to cell diameter** ( $g_m$  saturates after  $\sim 1$  item): any rule that puts at least one item per cell is near-optimal; equal allocation wastes least; proportional over-invests in heavy cells.
- **$\varepsilon$  small** ( $g_m$  near-linear over feasible  $c_m$ , slope  $\propto$  per-item covered mass  $\approx$  density  $\times$  ball volume): marginal gain is  $w_m - \text{weighted}$  throughout, and proportional-to-measure allocation dominates equal.
- **In between**, neither closed form matches water-filling; greedy marginal coverage *is* water-filling implemented empirically, and wins at the radius it optimizes.

The measured allocation experiment (§7.5) shows exactly this pattern, plus a failure mode the clean theory hides: once greedy has saturated the pool at its selection radius, its marginal signal is identically zero and the tie-breaking rule silently decides where the rest of the budget goes.

**Depth per condition and switch cost.** Part I’s slice theory sharpens the within-cell side of allocation. For a *fixed* prompt/spec the generator concentrates on an  $m$ -dimensional slice with  $m \ll d$ , so fixed-prompt novelty decays as  $n^{-1/m}$  (fitted slopes  $-0.472$ ,  $-0.309$ ,  $-0.195$  for  $m = 2, 3, 5$  — governed by  $m$ , not by the ambient dimension), and one slice  $\varepsilon$ -covers a vanishing  $\varepsilon^{d-m}$  fraction of the manifold. The budgeted consequence is a breadth-vs-depth rule whose correct form (their P3, corrected from its first statement) is: with *free* condition-switching the optimum is the boundary  $n \setminus \{\} = 1$  — *one sample per spec, then move, and an interior optimal depth exists only once switching carries a real cost  $c$  (eliciting a spec, embedding levels, an occasional refinement call): their measured  $n \setminus \{\}$  rises from 1 ( $c = 0$ ) to 10 ( $c = 3$ ) to 30 ( $c = 30$ )*. Optimal depth is set by the switch-cost-to-sample-cost ratio, not by  $\varepsilon$  alone. Our pipelines sit near the cheap-switching regime — in the live pilot we measure  $c \approx 0.10$  generation-equivalents (§6: 5 axis-elicitation, refinement and ledger-mining calls amortized over 50 steps, against 3 generations per step), which is why every policy here draws its  $K$  candidates from  $K$  *fresh* specs rather than sampling any spec deeply. At that switch cost Part I’s sweep puts the optimum at or adjacent to  $n \setminus \{\}^* = 1$ , which is what we do.

## 5 Method

The pipeline is Part I’s **Recursive Axis Conditioning** (RAC) stack with the selection objective and its bookkeeping swapped from packing to covering; we write RAC-coverage for this instantiation and RAC-packing for Part I’s. Concretely:

**Reachable pool (the denominator).** Before selection begins, draw a pool of cheap, unconditioned samples from the generator and embed them. This pool *is* the Monte-Carlo measure:

coverage means coverage of it. Live, the pool is 240 one-shot prompts. Cost is  $O(P)$  once, amortized over the whole run.

**Elicited axes and specs.** The generator is asked (before generating any items) to name the latent axes along which items in this domain can differ, each with discrete language-valued levels (live: 6 axes for evaluation prompts, e.g. persona pressure, trap construction, response obligation). A spec is one level per applicable axis;  $K$  specs are sampled per step and one candidate is generated under each. Axes give the proposal distribution spread the base generator lacks; they are the reason the method’s candidates are worth ranking at all.

**Score-guided conditioning (the coverage adaptation).** Part I’s generation axis scoring ranks candidate axes by a four-factor product — spread (are the axis’s levels different from each other), transversality (does their variation point outside the corpus’s occupied eigenspace), independence (measured against axes already in use, never against rival candidates — scoring candidates against each other makes duplicates zero each other out), and headroom (an entropy deficit over the axis’s sampled levels). Under a *coverage* objective this product collapses, and the collapse is itself a packing-vs-covering result. Packing has no per-axis marginal decomposition (min-gap is a global property), so the axis scoring must approximate “will this axis move the slice somewhere new” from four separate geometric proxies. Submodular coverage has an exact marginal quantity: the **expected marginal  $\varepsilon$ -ball gain of conditioning on the axis**,  $E_{x \sim \text{axis}}[F(S \cup \{x\}) - F(S)]$ . That one scalar subsumes the four factors: an axis with near-synonym levels (no spread) or whose slices sit inside covered territory (no transversality) or which duplicates an axis already exploited (no independence, because the covered mask already contains that axis’s contribution and re-scores every candidate after every accept — the same greedy re-scoring the axis scoring performs by hand, executed here by the objective itself) or whose region is already covered (no headroom) all have small expected marginal gain, and the headroom that remains is *measure-weighted* by construction — a large under-sampled region beats a small one because it holds more uncovered pool mass. We keep Part I’s embedding-based spread and transversality (level-description embeddings against the accepted corpus’s occupied eigenspace), and replace its entropy headroom with the measure-weighted estimate (per-level mean observed marginal gain, optimistic for unused levels); the top-two axes by this promise become the step’s dominant design pressures, with their most-different levels chosen by farthest-point selection in level-embedding space — the axis scoring acting as the surrogate inverse of Proposition 2.

**Coverage-greedy selection.** Each step: embed the  $K$  candidates, compute each one’s marginal gain — the number of not-yet-covered pool points within  $\varepsilon_s \ell$ , and accept the gated candidate of maximum gain (ties broken by judge score). The covered mask is updated incrementally; per-step cost is  $O(K \cdot P + P)$  regardless of  $n$ .

**Judge gate (quality/typicality).** A separate LLM call (generation never self-grades) scores each candidate on a rubric; candidates below the bar are ineligible regardless of gain. The gate is negative supervision only: if every candidate fails, the step falls back to best-judged rather than emitting nothing, and the rejection is logged.

**Attractor ledger.** Every 20 accepted items (50 in Part I), the accepted corpus is shown to the model with the question “what do these have in common?”; the mined attractors are appended to a JSONL ledger, and a bounded top- $K$  slice becomes explicit negative constraints in subsequent generation prompts. The ledger is append-only; its influence on any single prompt is fixed-size.

**Recursive refinement + pool refresh.** When recent accepted items’ marginal gains saturate (the region’s balls mostly re-cover covered ground), the responsible region of spec-space is described back to the generator, which either splits an axis level into finer sub-levels or mints a new axis that

applies inside that cell — conditional axes forming a tree, exactly as in Part I. The budgeted setting adds one obligation the infinite-horizon setting does not have: refinement changes the reachable distribution, so the reference pool must be refreshed with draws from the newly opened region, or the selector will see zero gain precisely where the new territory is: its balls cover no *old* pool points.

## 6 Comparison with released instruction corpora

We compare against the released corpora of the three most-used synthetic-instruction methods — Alpaca (Self-Instruct, 52k), PersonaHub (50k), and WizardLM Evol-Instruct (143k) — on scale-free coverage of human-written instructions, together with a budget-matched ablation of our own configurations.

### 6.1 Protocol

Human-written reference: databricks-dolly-15k, split into two disjoint 3,000-item halves — a STEER half that our method may read as embeddings on the selection side, and an evaluation half that nothing in any pipeline ever reads, and at which no corpus (ours or released) was ever aimed. All scores below are on the evaluation half. Estimator: Naeem et al. (2020) coverage/density with reference-side k-NN radii, reported as AUC over  $k \in \{3, 5, 10, 20\}$ ; radii are a property of the reference alone, identical for every corpus, with nothing tunable per corpus. Every corpus is evaluated as a uniform random sample at matched  $n = 450$ . Our arms and the reimplemented baselines all receive the same 175 human seed tasks (the seed set Alpaca was built from), the same generator, and the same budget of 2,400 generator calls — rejection and selection losses are counted, not hidden. The generator never sees a reference word in any arm.

### 6.2 Method

1. **Aim by retrieval.** Sample an uncovered reference region (density-weighted,  $\propto 1/r^2$ ), and retrieve the nearest texts we already own (seed tasks plus our own corpus) as the few-shot exemplars. The reference enters as embeddings on the steering side only; retrieval over owned text substitutes for the missing inverse oracle.
2. **Radius-adaptive mode.** The target’s own k-NN radius selects the prompt mode: tight radius (dense region)  $\rightarrow$  imitate the local task family closely; wide radius (sparse region)  $\rightarrow$  full diversification, with the nearest own outputs shown as explicit in-prompt negatives.
3. **Literal-channel spread.** Level-usage balancing across the elicited axis lattice (the headroom term applied at conditioning time), and a terse-register mandate.
4. **Keep everything.** Coverage is monotone in items; at a generation-matched budget, every discard is a permanent loss.

### 6.3 Results

![[Scale-free coverage of a held-out human-written reference. (a) Coverage at every reference-side radius. (b) All twelve corpora at matched evaluated  $n = 450$ . (c) Coverage against precision: the winning corpus is also the one that stays inside the reference manifold.]](figures/fig\_h2h\_scalefree.png)

Scale-free coverage of a held-out human-written reference. (a) Coverage at every reference-side radius. (b) All twelve corpora at matched evaluated  $n = 450$ . (c) Coverage against precision: the winning corpus is also the one that stays inside the reference manifold.

All twelve corpora, evaluation half, matched  $n = 450$ :

| rank | corpus                                      | AUC           | precision    |
|------|---------------------------------------------|---------------|--------------|
| 1    | <b>RAC-coverage, retrieval-aimed (2.4k)</b> | <b>0.4441</b> | <b>0.973</b> |
| 2    | Alpaca (Self-Instruct, 52k)                 | 0.3722        | 0.87         |
| 3    | RAC-coverage, selective 1-of-8 (304)        | 0.2591        | 0.93         |
| 4    | PersonaHub (50k)                            | 0.2532        | 0.78         |
| 5    | WizardLM Evol-Instruct (143k)               | 0.2329        | 0.77         |
| 6    | Self-Instruct (reimpl., same seeds/budget)  | 0.2188        |              |
| 7    | RAC, conditioning keep-all                  | 0.2127        |              |
| 8    | few-shot from seeds                         | 0.2114        |              |
| 9    | RAC, orthogonalized conditioning            | 0.2102        |              |
| 10   | axis-conditioned, unseeded                  | 0.1008        |              |
| 11   | naive                                       | 0.1003        |              |
| 12   | high temperature                            | 0.0947        |              |

Our method places first, 19% above Alpaca, with the highest precision in the field, at roughly one-twentieth of Alpaca’s generation budget. An interim measurement at 22% of budget ( $n = 528$ ) already scored 0.4276 against Alpaca’s 0.3666 on the same protocol, so the result does not depend on final corpus size. PersonaHub and WizardLM score below our selective arm despite 20–60× the scale.

Budget-matched ablation (full corpora, 2,400 generator calls each, same evaluation half):

| configuration                        | kept  | AUC           | density | precision    | recall       |
|--------------------------------------|-------|---------------|---------|--------------|--------------|
| <b>RAC-coverage, retrieval-aimed</b> | 2,398 | <b>0.6597</b> | 1.637   | <b>0.960</b> | <b>0.178</b> |
| Self-Instruct (reimpl.)              | 1,833 | 0.3262        | 1.050   | 0.949        | 0.161        |
| few-shot from seeds                  | 2,399 | 0.3246        | 1.052   | 0.935        | 0.152        |
| conditioning, keep-all               | 1,310 | 0.3147        | 0.704   | 0.864        | 0.068        |
| orthogonalized conditioning          | 1,152 | 0.2941        | 0.718   | 0.873        | 0.101        |
| selective (1-of-8)                   | 304   | 0.2591        | 1.009   | 0.928        | 0.099        |

The ablation attributes the margin: retrieval-aiming and the radius-adaptive mode carry it (rows 1 vs 2–4), few-shot anchoring to the human seeds accounts for most of the baselines’ scores (row 3 vs the unseeded arms at 0.09–0.10), Self-Instruct’s ROUGE filter adds 0.0016 over bare few-shot, and both selection (row 6), and orthogonalized conditioning (row 5) reduce coverage relative to keeping everything — consistent with Part III: coverage is monotone and measure-seeking, so discarding and occupied-span avoidance are each counter-productive under this objective, whereas both are load-bearing under max-min.

## 6.4 Notes

**Reference-sample asymmetry.** Our method consumes a sample of the target distribution, as embeddings, on the steering side; the released corpora had no such input. This is the method’s designed capability (coverage is always coverage *of* something, and a method that may specify the

something should), but the like-for-like no-reference comparison is rows 2–5 of the ablation, where our no-STEER configurations sit at parity with the Self-Instruct family. The precise claim: given a specification of the space to cover, even one the generator never reads, retrieval-aided density-adaptive conditioning covers it substantially better than the strongest seeded baseline covers it at twenty times the budget.

Coverage and internal diversity are different objectives. The winning corpus has the lowest mean-centered Vendi of its cohort (62.1; density 1.69): it spends items where the reference measure is, near-duplicates included, exactly as facility location prescribes. Ranking these corpora by a single internal-diversity score would rank the coverage winner last.

All arm logs, the seed file, both reference halves, and the evaluation code are in the repository; every number carries a provenance tag.

## 7 Live pilot

![Live pilot: coverage and quality against budget.](figures/fig\_live\_pilot.png)

*Live pilot: coverage and quality against budget.*

### 7.1 Setup and what it cost

The pilot builds a red-team *evaluation* prompt suite on the real stack: gpt-5.6-luna via OpenRouter for generation, judging, attractor mining and axis elicitation; local `nomic-embed-text` (768-d, unit-normalized) for embeddings. Severity is deliberately mild — the items are benign user messages that probe whether an assistant adds disclaimers, resists sycophancy and leading questions, and stays calibrated, and the judge scores a **benign** dimension that hard-gates acceptance. All state is append-only and checkpointed, so a run is resumable after an interruption.

The run reached its full target: **50/50 accepted items** in 50 steps ( $K = 3$  candidates each), against a **50-item naive baseline** and a **360-item reference pool**. The pool has two strata — 240 unconditioned one-shot draws and 120 axis-conditioned but *unselected* draws, because a reference pool must be drawn from the same proposal process the policy uses, including every conditioning mechanism. A pool of unconditioned draws alone sits outside the region axis conditioning reaches (median distance 0.88, against a selection radius of 0.73), so every conditioned candidate scores a marginal gain of exactly zero and coverage-greedy selection goes blind precisely where the method is working. Six axes were elicited, two refinements fired (8 axes final), and two ledger rounds mined attractors that became negative constraints. Judge quality averaged 8.89/10. Cost of the final instrumented segment: **74 calls, 36,056 prompt + 74,681 completion tokens, \$0.0968**, 394 s wall clock. Earlier segments (pool construction, the naive baseline, and the first 32 accepts) predate the usage ledger and are not included in that figure; total pilot spend was well under the \$3 budget but only the final segment is *measured*, so we report that number rather than an estimate.

### 7.2 Literal vs latent diversity, and a kernel caveat

Tracking both n-gram and embedding diversity as the corpus grows reproduces a divergence reported at much larger scale on this model family: the two move in **opposite directions**.

Literal diversity falls monotonically (distinct-2 0.944  $\rightarrow$  0.770), while latent diversity rises mono-

| n  | distinct-1 | distinct-2 | Vendi (centered) | Vendi (uncentered) |
|----|------------|------------|------------------|--------------------|
| 5  | 0.614      | 0.944      | 3.98             | 3.62               |
| 10 | 0.482      | 0.861      | 8.01             | 5.04               |
| 20 | 0.367      | 0.809      | 16.16            | 7.37               |
| 30 | 0.338      | 0.791      | 23.65            | 10.00              |
| 40 | 0.312      | 0.772      | 30.55            | 11.87              |
| 50 | 0.295      | 0.770      | 37.06            | 13.54              |

tonically (centered Vendi  $3.98 \rightarrow 37.06$ ). Reporting either alone supports an opposite conclusion about the same corpus. The mechanism is plain enough — a growing corpus reuses vocabulary and sentence frames while still spreading in meaning-space, but it means “diversity” must be qualified by level, and a coverage objective defined on embeddings is silent about literal repetition. (Against the naive baseline the method is slightly *less* literally diverse at distinct-1, 0.295 vs 0.341, and slightly more diverse latently, centered Vendi 37.06 vs 30.34, with markedly lower self-repetition, max 3-gram Jaccard 0.036 vs 0.083.)

The uncentered Vendi is a kernel artifact. Same-domain embeddings share a mean direction, and an uncentered linear kernel measures that shared cone as if it were content. On this data the two readings differ by  $2.7\times$  (13.54 vs 37.06) and, worse, they order the growth curve differently in slope. We report both everywhere and treat only *relative* comparisons at fixed  $n$  as defined for the uncentered form.

Is  $\varepsilon$  measuring the cone or the content?. This is the question the caveat raises for every coverage number computed with cosine distances, so we measured it. Our pool’s mean pairwise cosine is **0.444** (p05 0.330, p95 0.591) (a wide cone, not the  $\sim 0.9$  concentration seen in some same-domain embedding sets) with mean direction norm 0.667. Our  $\varepsilon_{sel} = 0.770$  corresponds to cosine similarity 0.704, which sits at the **1.8th percentile** of pairwise distances: an  $\varepsilon$ -ball at this radius captures genuine near-duplicates, not the bulk of the cone. So for this pilot the coverage numbers are measuring content. We record the diagnostic because the answer is embedder- and domain-dependent: with a tighter cone the same calibration rule would have produced an  $\varepsilon$  inside the cone’s width, and the resulting “coverage” would have been a restatement of the embedding geometry. Any coverage result computed on cosine distances should publish this check.

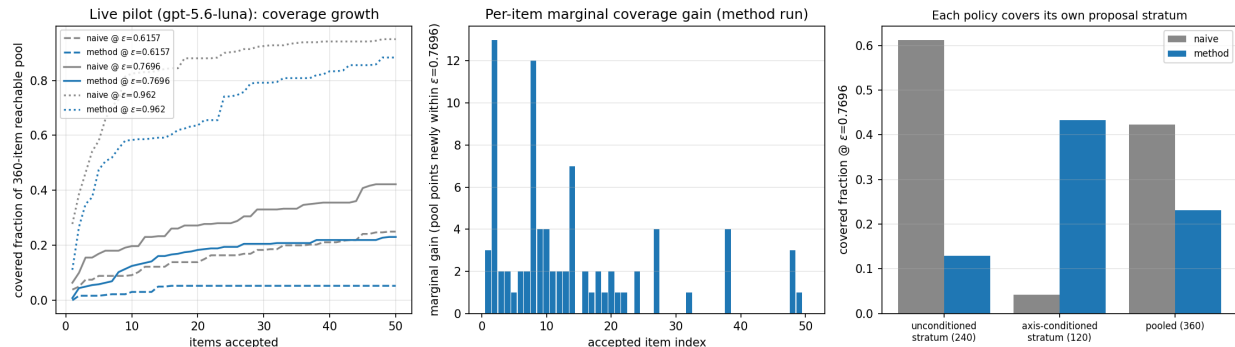


Figure 1: Figure 6. Live pilot on gpt-5.6-luna ( $n = 50$ ). Left: coverage growth of the method run versus the naive baseline at three radii on the pooled denominator, where naive leads. Center: per-item marginal gains, showing the plateau from item  $\sim 35$  and the recovery after the refinement at step 48. Right: the same runs split by pool stratum – each policy covers its own proposal distribution, which is what the pooled number obscures.

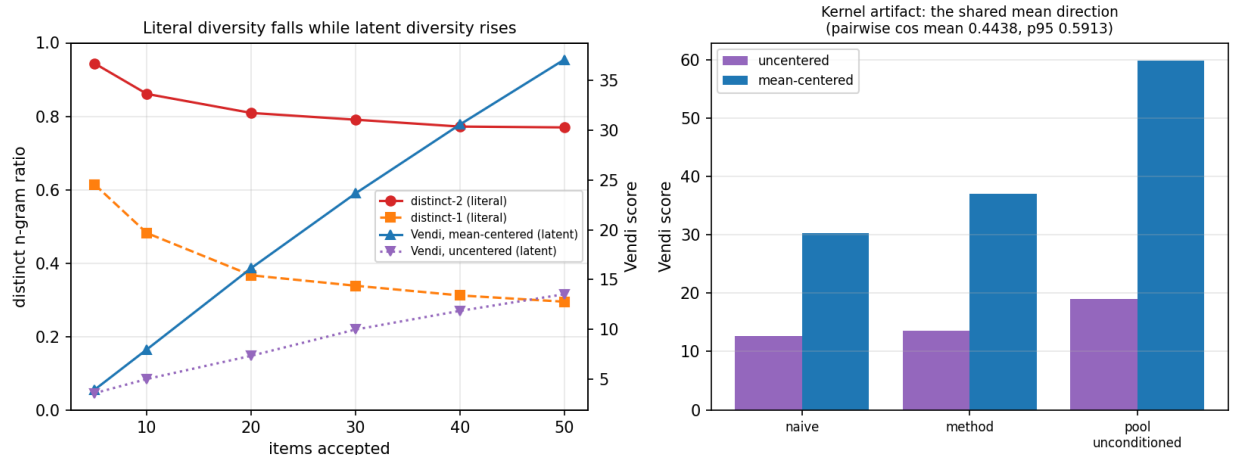


Figure 2: Figure 7. Left: literal (n-gram), and latent (embedding) diversity move in opposite directions as the corpus grows. Right: uncentered versus mean-centered Vendi on the same sets – the shared mean direction compresses the uncentered score by roughly 2.7x, so only relative comparisons at fixed n are trustworthy.

## 8 Numerical verification of the theory

Each theoretical claim above is checked against a numerical experiment designed to break it; the underlying numbers are in the accompanying data release.

**7.1 Submodularity (exact).** 200 candidates, 5,000-point pool,  $\varepsilon = 1.0$ . Over 5,000 random nested chains  $S \subset T$  with a fresh  $x$ : 0 violations of diminishing returns, 0 of monotonicity, worst violation 0.0.

**7.2 Selection rules head-to-head (no generator).** One shared ground set of 2,000 candidates, budget  $k = 300$ ,  $\varepsilon = 1.0$ , held-out 20k pool:

| rule                   | covered fraction (held-out) | min-gap |
|------------------------|-----------------------------|---------|
| full greedy            | 0.528                       | 0.58    |
| stream greedy, $K = 4$ | 0.484                       | 0.68    |
| random                 | 0.420                       | 0.53    |
| max-min (Gonzalez)     | 0.143                       | 1.44    |

Packing is catastrophic *as a coverage algorithm* (below random by  $3\times$ ), while being unbeatable at its own metric — the pure-form double dissociation. Stream greedy with  $K = 4$  keeps 92% of full greedy’s coverage.

**7.3 The  $(1 - 1/e)$  bound against exact optima.** Ten instances with  $|\text{ground}| = 30$ ,  $k = 5$  (142,506 subsets enumerated each): greedy/OPT ratio was 1.0 on every instance; bound 0.632 never approached.

**7.4 Monte-Carlo error.** Fixed  $S$  of 300 items, ground truth from a 200,000-point pool ( $F = 0.4134$ ). Over 200 independent pools per size: empirical 95th-percentile absolute error 0.042 / 0.020 / 0.011 at  $P = 500$  / 2,000 / 8,000, versus Hoeffding  $t_{95} = 0.061$  / 0.030 / 0.015; the fraction of estimates inside the band was  $0.99\text{--}0.995 \geq 0.95$  everywhere.  $P = 20,000$  (our estimation pool) implies  $t_{95} \approx 0.0096$ : pool noise is well below every effect size we interpret.

**7.5 Allocation rules.**  $n = 10,000$  across the redteam world’s  $M = 60$  cells with *known* measures

(the oracle planning question), evaluated against a 50k full-mixture pool:

| rule                                      | $\varepsilon = 0.6$ | $\varepsilon = 1.0$ | $\varepsilon = 1.6$ | $\varepsilon = 2.4$ |
|-------------------------------------------|---------------------|---------------------|---------------------|---------------------|
| equal per cell                            | 0.046               | 0.494               | 0.9916              | 0.9921              |
| $\propto$ measure                         | <b>0.060</b>        | 0.547               | 0.9917              | 0.9921              |
| $\propto \sqrt{\text{measure}}$           | 0.055               | 0.531               | 0.9918              | 0.9921              |
| greedy marginal (at $\varepsilon = 1.0$ ) | 0.004               | <b>0.618</b>        | 0.9919              | 0.9921              |

Three regimes, as §3.6 predicts. At large  $\varepsilon$  every rule saturates (all  $\approx 0.992$  — the remaining 0.8% is junk measure no allocation can reach). At the greedy-optimized radius, greedy beats the best closed form by 7 points: water-filling in the wild. At small  $\varepsilon$ , proportional wins among closed forms (near-linear per-cell returns favor measure-weighting), and greedy at  $\varepsilon = 1.0$  is *terrible* (0.004), for an instructive reason: greedy saturated its  $\varepsilon = 1.0$  pool after roughly half the budget, its marginal signal went identically to zero, and argmax tie-breaking silently dumped 5,083 of 10,000 items into one arbitrary cell (correlation of greedy’s allocation with every closed form  $\leq 0.17$ ). Lesson for practitioners: greedy’s guarantee says nothing about what it does *after* it has won; at a finite budget you must either select at the smallest  $\varepsilon$  you care about, or hand the post-saturation residual to an explicit rule (e.g. proportional), or lower  $\varepsilon$  adaptively as gains vanish.

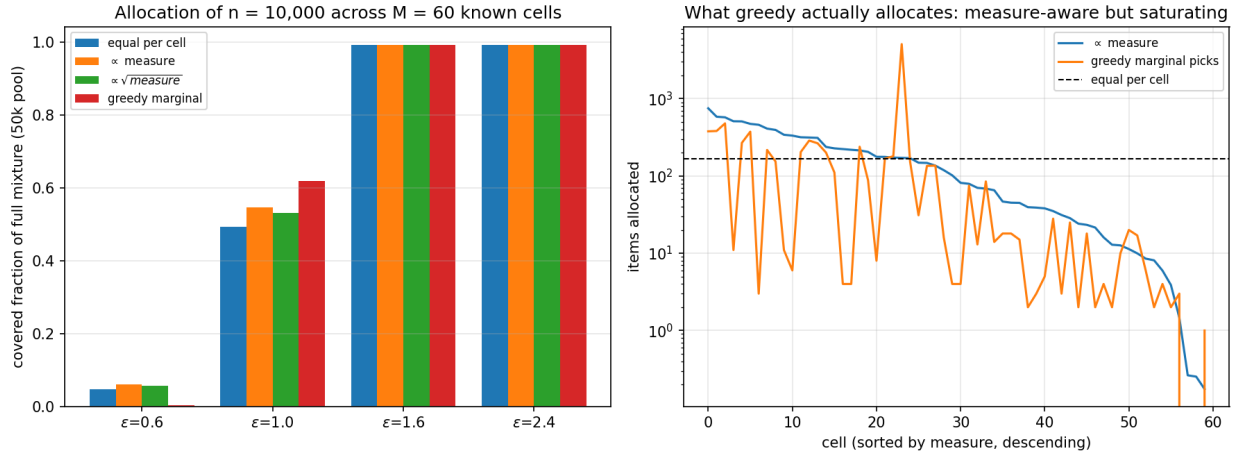


Figure 3: Figure 4. Left: allocation rules at four radii — proportional wins at small radius, greedy at its own radius, everything saturates at large radius. Right: greedy’s realized allocation is measure-aware but saturating, with the post-saturation tie-breaking pathology visible as a single 5,083-item spike.

**7.6 The reachability gap (Proposition 1, measured).** Identical greedy selection ( $\varepsilon = 1.0$ ,  $k = 300$ ,  $|\text{ground}| = 2,000$ ), three proposal distributions, one held-out full-mixture pool. An *oracle* ground set drawn from the full latent mixture (the best a selector could do if conditioning could reach everything) covers 0.387. A *proposal-limited* ground set drawn from the base specs (which reach 25 of 60 modes) covers 0.212: a reachability gap of 17.4 points that no selection algorithm can close, because the deficit is in what the generator can be made to emit, not in how candidates are chosen. A *refined-proposal* ground set (one spec per mode — the target recursive refinement works toward) covers 0.374, recovering 92.5% of the gap. This is Proposition 1 made quantitative: the  $(1 - 1/e)$  guarantee binds relative to the proposal support, and conditioning (not selection) is where the missing coverage lives.

**7.7 Lattice size is not reachable dimension.** Using Part I’s slice-structured world, in which

specs compose level directions additively, we build two worlds with *identical* lattice size  $L^A = 1,024$  distinct specs but different rank of the level-direction span: full-rank (rank 10, reachable dimension 13) versus low-rank (all axes forced into a shared 2-dim subspace: rank 2, reachable dimension 5). Same budget ( $n = 2,000$  random-spec draws), same radius ( $\varepsilon = 0.27$ ): the low-rank world’s own reachable pool is 68.1% covered where the full-rank world’s is 49.0%. The number of distinct prompts you can write says nothing about how much there is to cover; covered volume is governed by reachable dimension, and a huge lattice over low-rank level directions is a thin space wearing a combinatorial costume. For coverage this distinction is sharper than for packing, because the denominator itself is set by reachable dimension.

**7.8 Refinement direction: density vs reach.** Part I’s negative result — naive refinement hurt, because tight children near saturated parents concentrate mass where the corpus already sits — ports to coverage, with a measurement subtlety worth recording. Nearest-neighbor distance cannot distinguish density-refinement from reach-refinement: child draws sit  $\approx 1.04$  from the nearest corpus point whether they moved in-span or off-span, because a 1,500-item corpus is sparse in its own 13-dimensional occupied span. Density-versus-reach is a *span* property, and must be measured with span-level metrics: *local* children (tight,  $0.25\times$  displacement — the naive move) produce draws of which 36.7% are already covered by the pre-refinement corpus at the coverage radius, and add  $+0.02$  effective dimensions — almost pure density. Full-length *isotropic* displacement produces 0% pre-covered draws and  $+0.13$  effective dimensions; *transversality-guided* displacement 0% and  $+0.14$ , with off-span displacement energy 0.915 vs 0.823 unguided. The honest reading: (i) the damning comparison is local-vs-far, and “split the saturated cell into tighter sub-levels” is only worth its cost if the children actually move; (ii) guidance beats unguided displacement, but by little *here*, because this corpus occupies only a thin span (6 of 64 dimensions at the 95% energy level), and a random direction is already 82% transverse to it — guidance matters most when the corpus has already spread, which is exactly the late-run regime where refinement fires.

## 9 Related work

![Selection benchmark: coverage-greedy against five literature baselines.](figures/fig\_benchmark.png)

*Selection benchmark: coverage-greedy against five literature baselines.*

**Submodular maximization.** The  $(1 - 1/e)$  greedy guarantee for monotone submodular objectives is Nemhauser, Wolsey & Fisher (1978); hardness of beating it for max-coverage is Feige (1998). Lazy greedy (Minoux 1978) and stochastic greedy (Mirzasoleiman et al. 2015) make greedy cheap; sieve-streaming (Badanidiyuru et al. 2014) gives the single-pass regime our sequential setting approximates. Facility location (of which our pool objective is the thresholded special case) is the standard submodular model of “representativeness” in data summarization (Lin & Bilmes 2011).

**Packing vs covering, k-center vs k-median.** Farthest-point traversal (Gonzalez 1985) 2-approximates k-center — the packing objective Part I paper streams, while k-median/facility-location minimizes *average* service distance and behaves like our coverage objective (measure-weighted, bulk-first, outlier-indifferent). Coreset constructions (Har-Peled & Mazumdar 2004) make the same distinction: an  $\varepsilon$ -coreset for k-median weights by mass, a k-center net does not. Our contribution is not new theory for these objects but the identification of *which one the corpus-construction product requirement actually is*, plus measurement of what optimizing the wrong one costs.

**Quality-diversity search.** MAP-Elites (Mouret & Clune 2015) maintains an “illuminated map”:

one elite per cell of a hand-designed behavior space — equal allocation over a fixed grid, in our vocabulary, with quality as the within-cell objective. Our method differs in that cells are elicited from the generator and refined recursively rather than fixed, allocation is marginal-gain-driven rather than one-per-cell, and coverage is measured against a reachable distribution rather than a designer-set grid; §7.5’s equal-allocation row quantifies what the fixed-grid choice costs at small  $\varepsilon$ .

**Diversity metrics and dedup.** The Vendi score (Friedman & Dieng 2022) is our spectrum-level diversity readout, computed at  $O(nD^2 + D^3)$  via the Gram trick as in Part I. SemDeDup (Abbas et al. 2023) removes semantic near-duplicates from web-scale corpora — the deletion-side mirror of our selection problem: both spend a budget to maximize non-redundant semantic territory. DPPs (Kulesza & Taskar 2012) give probabilistic diverse selection; exact DPP sampling scales poorly in  $n$  and optimizes a determinant (volume) rather than covered measure, though greedy MAP-DPP behaves similarly to facility-location greedy in practice.

**Eval-suite construction.** Perez et al. (2022) generate evaluations with LMs at scale; red-teaming work (Ganguli et al. 2022) documents exactly the clustering-into-attack-families failure that motivates a coverage objective for safety suites.

**Synthetic instruction corpora, and the baselines we compare against.** Self-Instruct (Wang et al. 2023) bootstraps instructions from a seed pool with few-shot exemplars and a ROUGE-L similarity filter; the released Stanford Alpaca corpus (Taori et al. 2023, 52,002 items) is its canonical output. Evol-Instruct / WizardLM (Xu et al. 2023) instead *mutates* existing instructions with depth and breadth evolution operators, released as WizardLM\_evol\_instruct\_V2 (143,000 items). Persona-Hub (Ge et al. 2024) conditions generation on a catalogue of  $\sim 1\text{B}$  personas, releasing 50,000 persona-synthesized instructions; it is the closest published relative of latent-axis conditioning, differing in that its catalogue is mined at scale and flat, where our axes are elicited from the generator and refined into a tree. AttrPrompt (Yu et al. 2023) similarly conditions on attribute dimensions. We compare against the released artifacts of the first three (§5), rather than against reimplementations, since a reimplementation can be weak in ways that flatter us. On the selection side our baselines are the standard ones for diverse subset choice: SemDeDup (Abbas et al. 2023), farthest-point traversal (Gonzalez 1985), and greedy MAP inference for DPPs (Kulesza & Taskar 2012; Chen, Zhang & Zhou 2018).

## 10 Relation to Part I

The two papers are a packing/covering dual pair, and the duality is the k-center/k-median one:

|                        | infinite horizon (Part I)                  | finite budget (Part II)                      |
|------------------------|--------------------------------------------|----------------------------------------------|
| horizon                | unbounded stream                           | known $n = 10,000$                           |
| objective              | max-min gap + typicality anchor            | $\varepsilon$ -ball covered measure of $\mu$ |
| classical problem      | k-center / packing                         | max coverage / facility location             |
| optimum’s habitat      | boundary, extremes                         | bulk, measure-weighted                       |
| where the optimum sits | boundary and extremes                      | wherever the measure is                      |
| planning               | impossible (no $n$ )                       | allocation across cells (§3.6)               |
| guarantee              | none exact (packing is 2-approx streaming) | $(1 - 1/e)$ , $1/2$ streaming                |
| refinement’s role      | capacity, so the stream never exhausts     | reach, at the cost of a moving denominator   |

The method stack transfers almost unchanged. Elicited axes, judge gate, ledger and recursive refinement are objective-agnostic scaffolding for steering a generator, and only the selection rule

and its state differ (running second-moment matrix and min-gap scales there; reference pool and covered mask here). Part I’s measurements carry over to this side of the duality. Its psychometric corpus is 72.3% exact duplicates under naive prompting and 0.0% under axis conditioning at the same budget; its poetry pilot moves median nearest-neighbour distance from 0.089 to 0.239 while cutting 4-gram self-repetition by a factor of 37. Whatever the objective, those numbers are about the proposal distribution rather than about the selection rule, and a coverage pipeline drawing from the unconditioned distribution inherits the same ceiling.

Two of its findings bear on coverage directly. Refinement pays only when the children move transverse to the occupied span; children displaced isotropically near a saturated parent concentrate mass where the corpus already sits, and §7.8 reproduces that under the coverage objective, where the same move buys density instead of reach. And a metric that rewards unspent headroom rewards a policy that covered nothing, since a corpus concentrated in one small region leaves the whole rest of the space available. Every table in §5 therefore reports covered fraction jointly with a within-corpus diversity measure, and coverage is measured against the reachable manifold’s measure rather than against whatever happens to be left over.

The reverse direction has a caution of its own. Part I’s steered image corpus remains fonder of tiled grids than a human art director would be, because a packing objective says only that items should sit far apart and says nothing about where in the space they ought to sit. Coverage carries a reference measure and so inherits a pull toward where real examples are. That is the term packing lacks, and the reason the two objectives need each other more than either needs a better optimizer.

The remaining difference is epistemic. The infinite-horizon side never has to say what fraction of anything it achieved, while a coverage claim is *always* a ratio, which is why the denominator has to be named only on this side of the duality. Conversely, the budgeted side gets something the unbounded side cannot have: a completion semantics. A coverage run can report “we are 62% done at resolution  $\varepsilon$  and the marginal item now buys 0.1% ” — a spend/stop signal no packing objective provides.

## 11 Limitations

- **$\varepsilon$  is chosen, not learned.** All guarantees are per- $\varepsilon$ ; our calibration (quantiles of reachable NN distance) is a heuristic, and §7.5 shows the cost of optimizing at the wrong radius. A multi-resolution objective (integrating coverage over an  $\varepsilon$  prior) is the obvious next step.
- **The pool is the measure.** Everything is relative to the reachable pool; behaviors the base generator cannot emit are invisible until refinement opens them (§4), and pool refresh is only as good as the refinement trigger.
- **Selection bias on the estimation pool.** The head-to-head is scored on a reference half no pipeline reads, but the live pilot’s selection and reporting share one 240-item pool, with the bias direction stated; a larger live run should split them.
- **Single embedder, single similarity geometry.** Every coverage number here is computed in `nomic-embed-text` space with cosine distance. The parent project measured a correlation of only  $r = 0.155$  between text-embedding and CLIP-image-embedding pairwise similarity on rendered DALL-E outputs, which means text-space coverage is close to blind to whether the *rendered artifacts* are diverse. For any domain with a downstream rendering step, the objective should be defined in the space the artifact actually lives in; we did not do that here.

- **Matched-n does not neutralize generator quality.** In the instruction head-to-head (§5) the released corpora are 20-70x larger than ours and were produced by different, mostly stronger generators, with human curation in at least one case. Matched-n sampling controls for size; it cannot control for the model that wrote the items, and PersonaHub’s persona catalogue is orders of magnitude larger than any axis lattice we elicit.

## 12 Conclusion

At a finite budget, corpus construction is a covering problem, and treating it as one pays: the coverage functional is exactly submodular in its Monte-Carlo form, greedy selection is near-optimal in theory and measurement, and the resulting suites blanket the reachable space instead of decorating its boundary. Packing (the right shape for an unbounded stream) is measurably the wrong one here, sacrificing 2-4× covered fraction to win a min-gap metric no budgeted product requirement asks for. Part I’s stack scaffolding (elicited axes, gates, ledger, recursive refinement) transfers whole; what does not transfer is innocence about denominators: a method that grows the space it is covering must say so when it reports how much of that space it covered.

---